Óbudai Egyetem
Neumann János Informatikai Kar
Kiberfizikai Rendszerek Intézet

A LoRaWAN hálózatok biztonsági elemzése és sebezhetőségei tesztkörnyezetben

Security Analysis and Vulnerabilities of LoRaWAN Networks in a Testbed Environment

Diplomamunka

Szerző: Nagy Szabolcs
Neptun-kód: CGZNU9
Témavezető: Balázsné Dr. Kail Eszter
Konzulens: Bringye Zsolt
Intézet: Kiberfizikai Rendszerek Intézet
Képzés: Kiberbiztonsági mérnöki MSc

Budapest, 2026/2027. tanév

Tartalomjegyzék

Összefoglaló **3**

1. Bevezetés **5**

2. A LoRaWAN protokoll és biztonsági modellje **5**

2.1. Protokoll-architektúra 5

2.2. Biztonsági modell és kulcsarchitektúra 6

2.3. Ismert sérülékenységek a szakirodalomban 6

3. A tesztkörnyezet felépítése **9**

3.1. Hardver-komponensek 9

3.2. Szoftver-stack 9

3.3. A tesztkörnyezet elrendezése 10

4. Passzív rádiós lehallgatás SDR eszközzel **11**

4.1. Az eszköz és a pipeline 11

4.2. Eredmények 11

4.3. A sérülékenység értékelése 11

5. Fizikai SPI-busz tap – LoRaWAN keret kinyerése **12**

5.1. Elv és motiváció 12

5.2. Megvalósítás 12

5.3. FIFO-írás detekció 13

5.4. Eredmények 13

5.5. A támadási felület architekturális korlátai – integrált rádiós (STM32WL-osztályú) eszközök 14

6. AppKey brute-force – Join Request MIC alapján **15**

6.1. Elv 15

6.2. Implementáció 15

6.3. Eredmények 16

6.4. A teljes lánc hardveren, kulcs nélküli RF-outsiderként validálva 16

7. Firmware-bináris elemzés és álcázott csomópont **17**

7.1. Elv 17

7.2. Kulcskinyerés a binárisból 17

7.3. Álcázott csomópont generálása 18

7.4. Eredmények és hatásvizsgálat 18
7.5. Munkamenetkulcs-levezetés érvényesítése éles teszttel 18

8. Védelmi mechanizmusok értékelése 19
8.1. Firmware-szintű titkosítás 19
8.2. Biztonságos indítás (Secure Boot) 19
8.3. Megerősített firmware-konfiguráció 20
8.4. A debug-port lezárása olvasásvédelemmel (RDP) – hardveres igazolás . . . 20
8.5. A védelmi szint összefoglalása 21
8.6. Az OTA-frissítési csatorna hardeningje 22

9. Összefoglalás és jövőbeli kutatási irányok 25
9.1. Összefoglalás 25
9.2. Kiegészítő eredmények: hálózatszerver- és keretszintű vizsgálatok 26
9.3. Jövőbeli kutatási irányok 26

Összefoglaló

A LoRaWAN (Long Range Wide Area Network) protokoll az IoT-kommunikáció egyik legelterjedtebb megoldása kritikus infrastruktúrákban, okosvárosi rendszerekben és ipari környezetekben. A protokoll AES-128 alapú titkosítást és üzenethitelesítést alkalmaz, azonban a fizikai eszközök és a valós telepítések biztonsága számos ponton sebezhetőséget mutat.

Jelen dolgozatban bemutatjuk egy Óbudai Egyetemen felépített LoRaWAN tesztkörnyezet segítségével elvégzett biztonsági vizsgálat eredményeit. Négy különálló támadási forgatókönyvet valósítottunk meg és demonstráltunk: (1) passzív rádiós lehallgatás SDR eszközzel, (2) SPI-busz lecsapolás logikai analizátorral az MCU–rádióchip interfészen, (3) AppKey szótáras és kimerítő keresés a Join Request MIC alapján, valamint (4) firmware-binárisból automatizált hitelesítőadat-kinyerés és álcázott csomópont generálása.

A tesztkörnyezetben vizsgált LoRaWAN csomópont esetén az SPI-busz tap teljes plaintext hozzáférést adott a LoRaWAN keretekhez, a passzív sniffing pedig az összes Join Request metaadatát (AppEUI, DevEUI, DevNonce, MIC) feltárta hálózati hozzáférés nélkül. Az AppKey brute-force gyenge kulcsok esetén milliszekundumokon belül sikeres volt. Végül bemutattuk, hogy a firmware-szintű titkosítás bekapcsolása hatékonyan véd a közvetlen flash-dump alapú kulcskinyeréssel szemben.

Az eredmények rámutatnak arra, hogy az alkalmazott kriptográfiai védelem önmagában nem elégséges, és a fizikai biztonsági intézkedések, az erős kulcsmenedzsment, valamint a hardened firmware-konfiguráció együttes alkalmazása szükséges a valós fenyegetési modell kezeléséhez.

Kulcsszavak: LoRaWAN, IoT-biztonság, SPI-busz tap, SDR sniffing, AppKey, brute-force, firmware-elemzés, tesztkörnyezet, embedded security

Abstract. This paper presents the results of a security evaluation conducted on a LoRaWAN testbed built at Óbuda University. We demonstrate four attack scenarios against a commercial LoRaWAN end device: passive radio sniffing via SDR, physical SPI bus tap between the MCU and radio chip, AppKey recovery through dictionary and exhaustive brute-force of the Join Request MIC, and automated credential extraction from a firmware binary with rogue-node generation. Our findings show that the AES-128 cryptographic protection specified by the LoRaWAN standard is insufficient when physical access or weak key management is present. We also evaluate the effectiveness of firmware-level flash encryption as a mitigation.

Keywords: LoRaWAN, IoT security, SPI bus tap, SDR sniffing, AppKey, brute-force, firmware analysis, testbed, embedded security

1. Bevezetés

Az IoT (Internet of Things) eszközök robbanásszerű elterjedése az elmúlt évtizedben számos új biztonsági kihívást hozott magával. A LoRaWAN protokoll a kis fogyasztású, nagy hatótávolságú vezeték nélküli kommunikáció területén kiemelkedő szerepet tölt be: okos vízórák, gázdetektorok, ipari szenzorok és okosvárosi infrastruktúra-elemek tömegei kommunikálnak ezen a protokollon.

A LoRaWAN biztonsági modellje AES-128 alapú kriptográfiai védelmet nyújt mind az alkalmazásrétegen (AppSKey), mind a hálózati rétegen (NwkSKey). A Join mechanizmus ezenkívül friss munkamenetkulcsokat állít elő minden csatlakozásnál. Ennek ellenére a valós biztonsági kép összetettebb: a kulcsanyag a végpontok flash memóriájában tárolódik, az eszközök fizikailag hozzáférhetők, és az iparági tapasztalatok szerint a gyártók és integrátorok rendszeresen alkalmaznak gyenge vagy alapértelmezett kulcsokat.

Jelen diplomamunka az ÓE-NIK Kiberfizikai Rendszerek Intézetén felépített LoRaWAN tesztkörnyezet segítségével elvégzett biztonsági vizsgálat eredményeit mutatja be, módszertanában az iparági biztonsági felmérések gyakorlatához igazodva [**?**].

A dolgozat struktúrája a következő: a 2. fejezet a LoRaWAN protokollt és biztonsági modelljét ismerteti; a 3. fejezet a tesztkörnyezet felépítését mutatja be; a 4–7. fejezetek a négy demonstrált támadási forgatókönyvet tárgyalják; a 8. fejezet a védelmi mechanizmusokat értékeli; a 9. fejezet összefoglalja az eredményeket és jövőbeli kutatási irányokat javasol.

2. A LoRaWAN protokoll és biztonsági modellje

2.1. Protokoll-architektúra

A LoRaWAN egy csillag-topológiájú, aszimmetrikus kommunikációs protokoll, amelynek négy fő komponense van: végpont eszközök (end devices), átjárók (gateways), hálózati szerver (network server) és alkalmazásszerver (application server). Az átjárók LoRa modulált csomagokat vesznek és továbbítanak IP-alapú backhaulon a hálózati szerverre.

A LoRa fizikai réteg Chirp Spread Spectrum (CSS) modulációt alkalmaz, az SF (Spreading Factor) 7-től 12-ig változhat: nagyobb SF nagyobb hatótávolságot, de alacsonyabb adatátviteli sebességet jelent. Az EU868-as sávban a protokoll meghatározott csatornakészleten (868,1 MHz, 868,3 MHz, 868,5 MHz stb.) működik.

2.2. Biztonsági modell és kulcsarchitektúra

A LoRaWAN 1.0.x specifikáció szerint [**?**] minden eszköz rendelkezik:

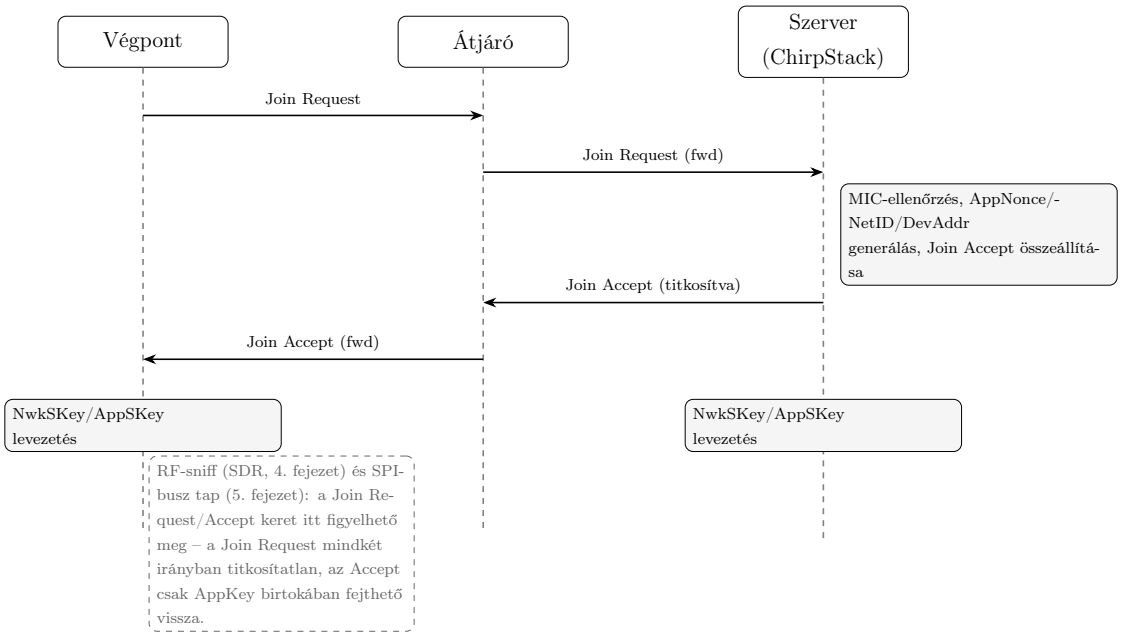
- **AppKey** (128 bites) – a gyökér szimmetrikus kulcs, amelyből a munkamenetkulcsok levezethetők,
- **AppEUI** és **DevEUI** – 64 bites azonosítók,
- **DevAddr** – 32 bites hálózati cím, amelyet a szerver oszt ki a Join eljárás során.

Az OTAA (Over-the-Air Activation) csatlakozási eljárásban az eszköz Join Request üzenetet küld, amelyet a szerver Join Accept üzenettel nyugtáz. A munkamenetkulcsok levezetése:

$$\text{NwkSKey} = \text{AES-128}(\text{AppKey},\ 0\text{x}01 \parallel \text{AppNonce} \parallel \text{NetID} \parallel \text{DevNonce} \parallel \text{pad}_{16}) \quad (1)$$

$$\text{AppSKey} = \text{AES-128}(\text{AppKey},\ 0\text{x}02 \parallel \text{AppNonce} \parallel \text{NetID} \parallel \text{DevNonce} \parallel \text{pad}_{16}) \quad (2)$$

Az adatkeretek payloadja AppSKey-jel titkosított (AES-128-CTR), integritásukat NwkSKey-jel számolt MIC védi.



1. ábra. Az OTAA Join-folyamat üzenetváltása és a két passzív megfigyelési pont (Végpont oldali SPI-tap, illetve a rádiós link SDR-sniffelése).

2.3. Ismert sérülékenységek a szakirodalomban

A LoRaWAN biztonsági modelljének gyengeségeit több független kutatás is vizsgálta [**?** **?** **?** **?** **?**]. Az alábbiakban a dolgozat szempontjából releváns, ismétlődően felmerülő

problémákat tekintjük át; a 4–7. fejezetben bemutatjuk, melyik ismert probléma az, amit a saját tesztkörnyezetben is sikerült demonstrálni, és melyik marad ezen dolgozat keretein kívül eső, pusztán elméleti kockázat.

Replay-támadás (DevNonce-újrafelhasználás). A LoRaWAN 1.0.x szabványban a Join Request DevNonce mezője mindössze 16 bites, véletlenszerű érték [**?**]. A specifikáció előírja, hogy a Join Server tartsa nyilván az adott eszköztől már látott DevNonce-értékeket, és utasítsa el az ismétlődőket – azonban ez az implementációk minőségén múlik, és több kereskedelmi hálózati szerver-megvalósításban hiányosan volt implementálva vagy megkerülhető volt [**?**]. Ha egy korábban rögzített, érvényes Join Request visszajátszható, az minden esetben új munkamenetkulcs-párt (NwkSKey/AppSKey) vált ki a szerver oldalán – ez önmagában nem ad hozzáférést a kulcsokhoz, de szolgáltatás-megtagadást (a legitim eszköz munkamenete érvénytelenítődik) vagy a hálózati napló megzavarását teheti lehetővé. A Join-eljárás formális biztonsági elemzése és konkrét, kombinált visszajátszás-alapú támadási forgatókönyvek is megerősítik, hogy ez a felület a gyakorlatban is kihasználható, nemcsak elméletben [**?** **?** **?** **?**].

Bit-flipping és a csonkított MIC. A payload titkosítása AES-128 CTR (counter) módban történik, ami – minden stream cipherhez hasonlóan – malleábilis: a titkosított szöveg egy bitjének megváltoztatása a visszafejtett plaintext ugyanazon bitjét változtatja meg, anélkül, hogy a titkosítás önmagában ezt jelezné [**?**]. Ismert (vagy kitalálható) plaintext-struktúra esetén (pl. fix formátumú szenzoradat-keretek) ez célzott payload-manipulációt tesz lehetővé. Az integritásvédelem a 128 bites AES-CMAC-ból számolt, de a keretben csak 4 bájtra (32 bitre) csonkított MIC – egy vak, találgatáson alapuló hamisítás sikerességi esélye emiatt 2^{-32} próbánkénti keretre adódik, ami lényegesen alacsonyabb biztonsági margót jelent, mint egy teljes 128 bites MAC-nál, és a szakirodalomban rendszeresen felmerülő kritikai pont [**?**].

ACK spoofing. Megbízhatósági (confirmed) uplink keretek esetén az eszköz egy hálózati oldali ACK-ra vár nyugtázásként. Mivel az ACK a letöltési (downlink) ablakban érkezik, egy támadó – aki a gateway és a hálózati szerver közti csatornát, vagy magát az RF-linket kontrollálja – hamis ACK-ot injektálhat, ami azt a látszatot kelti az eszköznek, hogy az üzenet célba ért, miközben az valójában sosem jutott el a szerverig [**?**]. Ez különösen kritikus, ha az alkalmazás logika a sikeres kézbesítést biztonsági- vagy vészjelzés nyugtázásaként kezeli – hasonló ember-a-középen (MITM) forgatókönyveket valós tesztkörnyezetben is demonstráltak [**?** **?**].

RF jamming és szolgáltatás-megtagadás. Az EU868 sávban a LoRaWAN egy rögzített, kis méretű csatornakészleten (jellemzően 8 alap csatorna, 125 kHz sávszélesség) működik, szigorú duty cycle-korláttal (1% csatornánként) [**?**]. A fix csatornakészlet és a korlátozott adóteljesítmény miatt egy célzott zavaró jel viszonylag olcsón és energiahatékonyan tud teljes területi lefedettségű szolgáltatás-megtagadást okozni – ezzel szemben a szabvány önmagában nem nyújt védelmet (frekvencia-ugratás vagy spread-spectrum ellenállás korlátozott a rögzített csatornaterv miatt). Kereskedelmi hardverrel épített szelektív és reaktív zavaró eszközökkel ezt a sebezhetőséget valós méréssel is igazolták, és a fizikai rétegbeli LoRaWAN-támadások szisztematikus áttekintése is a jamminget sorolja a legkönnyebben kivitelezhető fenyegetések közé [**? ? ?**].

ABP-aktiváció és a forward secrecy hiánya. Az ABP (Activation By Personalization) aktivációs módnál nincs Join-folyamat: a DevAddr és a két munkamenetkulcs (NwkSKey, AppSKey) statikusan, gyártáskor kerül az eszközbe, és soha nem cserélődik, amíg valaki manuálisan le nem cseréli őket [**?**]. Ez egyrészt megszünteti a DevNonce-alapú replay-védelmet (nincs újra-Join, amit védeni kellene), másrészt közvetlenül összefügg a CTR-mód keystream-újrafelhasználási kockázatával: ha egy eszköz keretszámlálója (FCnt) bármilyen okból (reset, firmware-flash, óraátállás) egy már korábban használt értékre áll vissza, a payload-titkosítás keystreamje ismétlődik, ami – ismert plaintext birtokában – a titkosítás teljes megkerülését teszi lehetővé.

Fizikai rétegbeli ujjlenyomat-vétel, mint detekciós ellenintézkedés. A fenti támadások jelentős része (replay, ACK spoofing, rogue node) közös tulajdonsága, hogy a hamis keret a protokoll szintjén hitelesnek tűnhet, ha a támadó ismeri (vagy visszajátssza) a szükséges azonosítókat. Robyns és szerzőtársai megmutatták, hogy az adók rádiófrekvenciás lenyomata (a jelalak apró, hardverfüggő torzításai) felügyelt és zero-shot tanulási módszerekkel megkülönböztethető, így elméletileg lehetőség nyílik egy klónozott vagy rogue eszköz protokoll-szinten észrevehetetlen kommunikációjának fizikai rétegen történő felismerésére [**?**]. Ez a dolgozat 7. fejezetében bemutatott álcázott csomóponttal szemben releváns, de jelenleg nem megvalósított védelmi irány. Újabb kutatások azt is megmutatták, hogy ez a fizikai ujjlenyomat-vétel maga is támadható: ellenséges (adversarial) perturbációval megtévesztő vagy álcázó jelalak állítható elő mélytanulás-alapú detektorok ellen, ami a védelmi irány hosszú távú megbízhatóságát önmagában is megkérdőjelezi [**? ?**].

A LoRaWAN 1.1-es specifikáció [**?**] a fenti problémák egy részét kezeli: a DevNonce-t 16 bites véletlen értékről monoton növekvő számlálóra cseréli (kezelve a replay-kockázatot resetelő eszközöknél is), bevezeti a JoinEUI-t (az AppEUI kiterjesztése/átnevezése), és

a NwkSKey-t három különálló kulcsra bontja (SNwkSIntKey, FNwkSIntKey, NwkSEncKey), csökkentve egy esetleges kulcsszivárgás hatókörét. A piacon – így a jelen tesztkörnyezetben vizsgált eszköz esetén is – azonban túlnyomórészt még 1.0.x eszközök vannak forgalomban, ezért a dolgozat fókusza is az 1.0.x biztonsági modellen marad.

3. A tesztkörnyezet felépítése

3.1. Hardver-komponensek

A tesztkörnyezet az alábbi komponensekre épül:

1. táblázat. A tesztkörnyezet hardver-elemei

Komponens	Eszköz	Szerepkör
Végpont csomópont	kereskedelmi LoRaWAN node	OTAA kliens
Átjáró	kereskedelmi LoRaWAN GW	LoRaWAN GW
Hálózati szerver	ChirpStack v4	szoftver szerveren
Passzív sniffer	SDR vevő + antenna	rádiós megfigyelés
Logikai analizátor	logikai analizátor	SPI-busz tap

A vizsgált LoRaWAN csomópont egy integrált MCU + rádióchip megoldáson alapul. Az MCU és a rádióchip közötti SPI-busz pinjai hozzáférhetők a PCB-n, ami lehetővé teszi a fizikai lehallgatást. Alapértelmezetten a firmware-szintű titkosítás és a biztonságos indítás nincsenek bekapcsolva.

3.2. Szoftver-stack

A végpontokon nyílt forráskódú LoRaWAN firmware stack fut. A hálózati szerver ChirpStack v4.19.0 [**?**], konténerizált (docker-compose) architektúrában futtatva:

2. táblázat. A hálózati szerver oldal szoftver-komponensei

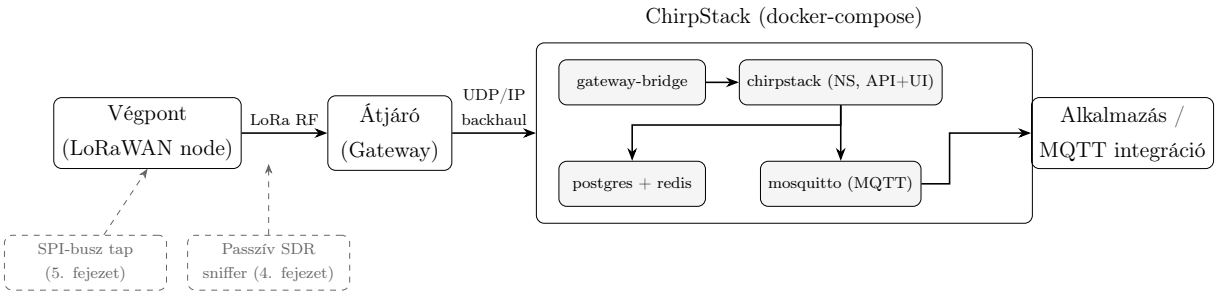
Szolgáltatás	Image	Szerepkör
`chirpstack`	`chirpstack/chirpstack:4`	LoRaWAN network server (API+UI)
`chirpstack-gateway-bridge`	`chirpstack/chirpstack-gateway-bridge:4`	Gateway↔NS packet forwarder híd
`chirpstack-rest-api`	`chirpstack/chirpstack-rest-api:4`	REST/JSON ↔ gRPC gateway
`postgres`	`postgres:14-alpine`	ChirpStack adatbázis
`redis`	`redis:7-alpine`	ChirpStack cache/session
`mosquitto`	`eclipse-mosquitto:2`	MQTT broker (integráció, uplink-publikálás)

A szerverhez távoli hozzáférés egy dedikált, nem-admin operációs fiókkal történik, amely kizárólag a projektmappára és a Docker-parancsok futtatására jogosult – ez tudatos, minimál jogosultságú hozzáférés-tervezési döntés, nem az alapértelmezett adminisztrátori profil használata.

A passzív snifferhez GNU Radio alapú pipeline-t és LoRa dekódoló modult alkalmazunk.

3.3. A tesztkörnyezet elrendezése

A laboratóriumi elrendezés biztosítja, hogy az összes támadási forgatókönyv kontrollált és megismételhető körülmények között hajtható végre. A végpontok EU868-as sávon, SF7, BW125 kHz, CR4/5 paraméterekkel kommunikálnak.



2. ábra. A tesztkörnyezet architektúrája: a végponttól a hálózati szerveren át az alkalmazásig, a két passzív támadói megfigyelési ponttal (szaggatott).

4. Passzív rádiós lehallgatás SDR eszközzel

4.1. Az eszköz és a pipeline

A passzív sniffinghez egy olcsó, széles körben elérhető SDR (Software Defined Radio) vevőt alkalmaztunk. A LoRaWAN EU868-as sávon 868,1 MHz-en szinkronizáltunk.

A GNU Radio pipeline az alábbi lépésekből áll:

1. SDR forrás: 868,1 MHz center, megfelelő mintavételi frekvencia
2. Decimáció a LoRa sávszélességre
3. LoRa dekódoló modul: frame sync, FFT demodulátor, Gray-leképezés, deinterleaver, Hamming-dekódolás, CRC-ellenőrzés
4. LoRaWAN PHY parser: MHDR értelmezés, frame-típus azonosítás

4.2. Eredmények

Join Request keretek valós idejű dekódolása sikeresen megvalósult. Minden Join Request esetén az alábbi adatok nyílt szövegben kinyerhetők:

```
Type:      Join Request
AppEUI:    06cfd4758b9da524
DevEUI:    3a26eef4b5898ccd
DevNonce:  8e62
MIC:       f9138d2f
```

1. Listing. Dekódolt Join Request mezők – passzív sniffing

A Join Accept a szerver által küldött, titkosított üzenet, amelynek tartalma AppKey nélkül nem fejthető vissza. Az adatkeretek payloadja szintén titkosított. Azonban az összegyűjtött AppEUI/DevEUI/DevNonce értékek önmagukban is hasznos információt jelentenek: eszközazonosítók, szervezeti prefixek (OUI), csatlakozási minták válnak megfigyelhetővé.

4.3. A sérülékenység értékelése

A LoRaWAN specifikáció szándékosan nem titkosítja a Join Request keretet, mivel a szerver az AppKey nélkül nem tudná ellenőrizni a MIC-et. A metaadatok (AppEUI, DevEUI) kitettsége az eszközazonosítás és célzott támadások szempontjából kockázatot

jelent. A LoRaWAN 1.1-es verziója részben kezeli ezt a problémát a JoinEUI bevezetésével és a DevNonce tartomány bővítésével, de teljes metaadat-titkosítást nem nyújt.

5. Fizikai SPI-busz tap – LoRaWAN keret kinyerése

5.1. Elv és motiváció

A vizsgált rádióchip nem rendelkezik saját kriptográfiai motorral – az MCU firmware által összeszerkesztett LoRaWAN keretet már kész binárisként írja a rádióchip FIFO regiszterébe SPI-n keresztül. Következésképpen az SPI-buszon a LoRaWAN keret *plaintext* formában jelenik meg közvetlenül az adás előtt, akkor is, ha a firmware-szintű titkosítás aktív – az csak a flash-tárolt adatot védi, nem a memóriában lévő futásidejű adatot. Hasonló, fizikai oldalcsatorna-alapú kulcs-visszanyerési technikákat valós LPWA végpontokon már korábban is demonstráltak, megerősítve, hogy a futásidejű adatút az AppKey-védelem gyakorlati gyenge pontja lehet, függetlenül a flash-szintű titkosítástól [**?** **?**].

Join Request esetén a keret teljesen titkosítatlan:

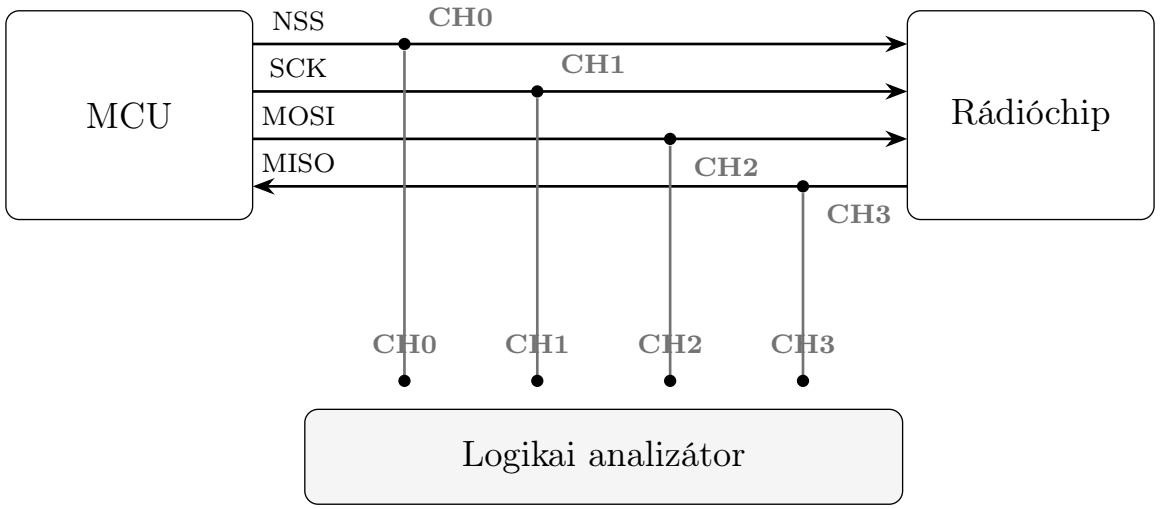
$$\text{Frame} = \text{MHDR} \parallel \text{AppEUI} \parallel \text{DevEUI} \parallel \text{DevNonce} \parallel \text{MIC} \quad (23\ \text{bájt}) \tag{3}$$

5.2. Megvalósítás

A logikai analizátor csatornáit az alábbi pin-kiosztás szerint csatlakoztattuk az MCU–rádióchip SPI-interfészre:

3. táblázat. Pin-kiosztás – SPI-busz tap

CH	SPI jel	Leírás
CH0	NSS	Chip Select
CH1	SCK	Clock
CH2	MOSI	Master→Slave
CH3	MISO	Slave→Master



3. ábra. Az MCU–rádióchip SPI-busz pin-kiosztása és a logikai analizátor csatornáinak hozzárendelése (CH0=NSS, CH1=SCK, CH2=MOSI, CH3=MISO) – minden csatorna egyenes, kereszteződésmentes vonallal a saját jeléhez kötve.

Az SPI-protokoll paraméterei: CPOL=0, CPHA=0, MSB first, CS active low. A szükséges mintavételi frekvencia a rádióchip SPI-sebességének legalább kétszerese (jellemzően $\geq$20 MHz).

5.3. FIFO-írás detekció

A rádióchip SPI-kommunikáció első bájtja kódolja a műveletet:

- byte & 0x80 = 0x80: írás
- byte & 0x7F: regisztercím

A RegFifo (0x00) írása (`0x80`-as első bájt) tartalmazza a LoRaWAN keretet. Tipikus TX-sorozat:

```
NSS low  0x8D 0x80  NSS high   ; FifoTxBaseAddr = 0x80
NSS low  0x8E 0x80  NSS high   ; FifoAddrPtr = 0x80
NSS low  0x80 [23 bytes]        ; FIFO write  <- LoRaWAN keret itt
NSS low  0xA2 0x17  NSS high   ; PayloadLength = 23
NSS low  0x81 0x83  NSS high   ; OpMode = TX
```

2. Listing. SPI TX sorozat – Join Request adás előtt

5.4. Eredmények

A FIFO-írás sikeresen detektálható volt, a teljes Join Request keret kiolvasható volt:

```
FIFO WRITE #1  [tranzakcio #312]
PHY (23 B): 0024a59d8b75d4cf06cd8c89b5f4ee263a628ef9138d2f
Tipus:    Join Request
AppEUI:   06cfd4758b9da524
DevEUI:   3a26eef4b5898ccd
DevNonce: 8e62
MIC:      f9138d2f
```

3. Listing. SPI-busz tap eredménye

A kinyert AppEUI és DevEUI megegyezett a szerveren regisztrált értékekkel, megerősítve a támadás sikerét. Firmware-szintű titkosítás bekapcsolt állapota esetén a flash-dump plaintext kulcsot nem ad vissza, de az SPI-tap a futásidejű adatot rögzíti, így ez a védelmi intézkedés az SPI-tap fenyegetésével szemben csak részleges.

A mérést egy 20 MHz-es mintavételi frekvenciájú, 33 554 432 mintás (kb. 1,68 másodperces) capture-ablakon végeztük, amely 571 SPI-tranzakciót tartalmazott; ebből egyetlen tranzakció volt FIFO-írás (a fenti Join Request), ami megfelel az elvárt viselkedésnek – a rádióchip csak adás előtt írja a FIFO-t, a többi tranzakció regiszter-konfiguráció és állapotlekérdezés.

Megjegyzés a chip-lefedettségről. Az eszköz-oldali kinyerő implementáció a vizsgált SX1276-alapú (V2) rádiómodulon túl előkészítve tartalmazza az SX1262-alapú (V3/V4) csomópontok támogatását is (más FIFO-írás parancskódolással). Ez az út egységteszteken és szintetikus, kézzel összeállított Join Request mintán helyesen működik, de – mivel nem állt rendelkezésre valódi SX1262-alapú hardveres capture – **valós hardveren nem lett ellenőrizve**, ezért az eredmények közé itt tudatosan nem került be: a dolgozat kizárólag a ténylegesen hardveren igazolt SX1276/V2 méréseket közli önálló eredményként.

5.5. A támadási felület architekturális korlátai – integrált rádiós (STM32WL-osztályú) eszközök

A fenti SPI-tap támadás előfeltétele, hogy az MCU és a rádiócsip két önálló lapka legyen, közöttük kívülről hozzáférhető SPI-busszal – ez a vizsgált, viszonylag olcsó Heltec-node-ok (ESP32 + külső SX1276/SX1262 rádiócsip) esetén fennáll. A tesztkörnyezetben emellett egy jóval drágább, STM32WL55 MCU-alapú ipari fejlesztőpanel (NUCLEO-WL55JC1) is rendelkezésre állt, amelyen a LoRa rádió a mikrokontroller lapkájába van integrálva – nincs önálló, kívülről hozzáférhető rádiócsip, így nincs lehallgatható külső MCU–rádiócsip SPI-busz sem.

Ez azt jelenti, hogy a jelen fejezetben bemutatott támadási felület a STM32WL-osztályú eszközökön architekturálisan nem létezik, függetlenül bármilyen firmware-szintű hardeningtől – nem egy konfigurálható védelem szünteti meg, hanem maga a lapka-integráció zárja ki eleve. (A STM32WL-en egyébként is azonosított, debug-port-alapú fizikai támadási felületet a 8. fejezet külön tárgyalja.) Ez önmagában egy dokumentálható biztonsági-gazdasági következtetés, új mérés nélkül: az olcsóbb, kéttagú (MCU + külső rádiócsip) node-architektúra extra fizikai támadási felületet nyit, amit a drágább, integrált rádiós platform megválasztása önmagában, mérés nélkül is kizár – a SPI-tap fenyegetés szempontjából tehát a magasabb hardverköltség közvetlenül magasabb fizikai biztonsági szintet jelent.

6. AppKey brute-force – Join Request MIC alapján

6.1. Elv

A Join Request MIC számítása:

$$\text{MIC} = \text{AES-128-CMAC}(\text{AppKey},\ \text{MHDR} \parallel \text{AppEUI} \parallel \text{DevEUI} \parallel \text{DevNonce})[0:4] \quad (4)$$

Mivel a MIC inputja (MHDR, AppEUI, DevEUI, DevNonce) a levegőből passzívan rögzíthető (ld. 4. fejezet), egy támadó az összes AppKey-jelöltet offline, hálózati hozzáférés nélkül tesztelheti: minden jelöltre kiszámítja az AES-128-CMAC-ot, és összehasonlítja az utolsó 4 bájttal. Ugyanezt az elvet nyilvános, offenzív biztonsági eszközök (pl. a LoraPWN és a Loracrack projektek) is alkalmazzák gyenge/alapértelmezett AppKey-ek ellen, megerősítve, hogy ez nem elméleti, hanem a gyakorlatban is aktívan kihasznált támadási út [**? ?**].

6.2. Implementáció

Az eszköz három módot támogat:

1. **Wordlist mód:** ismert gyenge/alapértelmezett kulcsok listája (gyártói alapértelmezések, nulla kulcs, DEADBEEF… mintás kulcsok).
2. **Szekvenciális mód:** az összes legfeljebb N darab nemnulla bájtot tartalmazó 128 bites kulcs kimerítő keresése.
3. **Hybrid mód:** wordlist először, majd szekvenciális keresés (alapértelmezés).

A belső ciklus Python `cryptography` könyvtár AES-128-CMAC implementációját alkal-

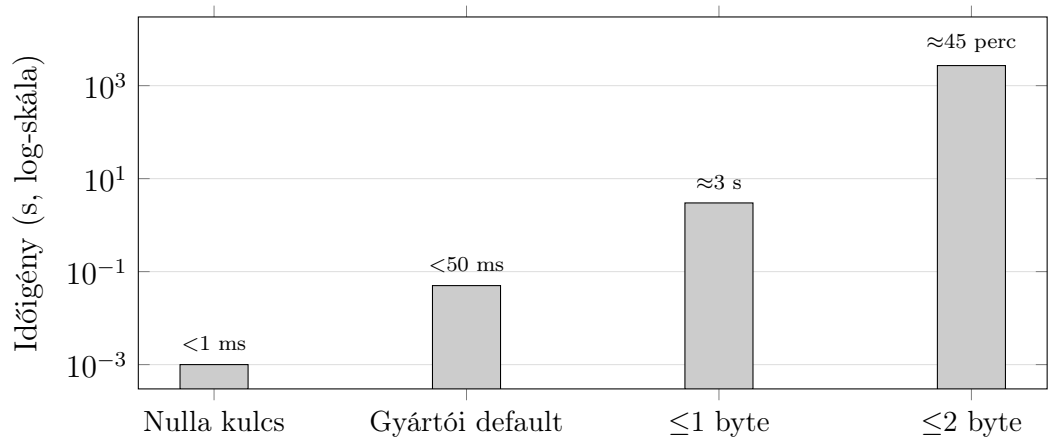
mazza; egyetlen magos futás kb. 10^6 kulcsot/s sebességen tesztel.

6.3. Eredmények

4. táblázat. AppKey brute-force teszteredmények

Kulcs-típus	Mód	Időigény
`00:00:...:00` (nulla kulcs)	wordlist	< 1 ms
Gyártói alapértelmezés	wordlist	< 50 ms
Legfeljebb 1 nemnulla bájt	szekvenciális	≈ 3 s
Legfeljebb 2 nemnulla bájt	szekvenciális	≈ 45 perc

A teszt során a nulla kulcsot és több ismert gyártói alapértelmezést sikeresen visszaállítottunk. Erős, véletlenszerű 128 bites kulcsok esetén a brute-force nem kivitelezhető a jelenlegi számítási kapacitással.



4. ábra. Az AppKey brute-force időigénye kulcstípusonként (log-skála; a mért értékek az előző táblázattal egyeznek).

6.4. A teljes lánc hardveren, kulcs nélküli RF-outsiderként validálva

A fenti offline crackert egy teljes, kulcs nélküli RF-outsider forgatókönyvben is végigvittük: a támadó kizárólag a levegőből fogott adatokkal dolgozik, sem hálózati (API/MQTT), sem fizikai (SPI/debug-port) hozzáférése nincs az áldozathoz.

Az áldozat egy Heltec V3 panel, szándékosan gyenge, a gyakorlatban széles körben elterjedt tutorial-kulccsal (`0102030405060708090A0B0C0D0E0F10`) felprogramozva, amely bootolás után folyamatosan OTAA Join Requestet küld. A független RF-támadói eszköz

egy második panel (Heltec V4), amely saját rádiójával, nyers LoRa-vételi módban, a Join-csatornán (868,1 MHz, SF9, BW125) figyel – ez a csatorna/SF-pár egy korábbi, egyszeri instrumentált megfigyelésből derült ki, a tényleges crack-láncban már nincs rá szükség. A V4 kilenc Join Requestet fogott el a levegőből (RSSI ≈ -120 dBm, tehát valódi, gyenge RF-vétel, nem vezetékes lehallgatás), növekvő DevNonce-értékekkel.

A `appkey_crack.py` eszköz mind a kilenc, kizárólag RF-en elfogott kereten ugyanarra a gyenge kulcsra tört – a crack-lépés tehát bizonyítottan nem igényel semmilyen, az RF-vételen túli információt. Kontrollként egy valódi, véletlenszerű 128 bites kulccsal felprogramozott eszközön (ld. fent, a keylessoutsider-sweep részeként, 9. fejezet) ugyanez a lépés elbukik – a támadás tehát bizonyítottan gyengekulcs-specifikus, nem a LoRaWAN Join-eljárás vagy a MIC-séma hibája.

A *weaponizálás* lépésben a feltört kulcsot a hálózati szerverbe is regisztráltuk, és egy újabb, valódi Join-cserét figyeltünk meg. A sniffelt+feltört anyagból (titkosított Join Accept, DevNonce) levezetett NwkSKey és AppSKey **bitre megegyezett** a hálózati szerver belső munkamenet-állapotával. A levezetett AppSKey-jel egy valódi, az áldozat által később küldött titkosított uplink FRMPayload-ját is sikeresen visszafejtettük, az eredeti (`0xA5`) payload-értéket helyreállítva – ez a bizalmasság teljes körű megtörését demonstrálja, a NwkSKey birtokában pedig a keret-hamisítás lehetősége is adott.

Severity-keretezés: ez a lánc a célpont kulcskezelési hibáját (gyenge/alapértelmezett AppKey) aknázza ki, nem a LoRaWAN-protokoll vagy a ChirpStack kriptográfiájának hibáját – rendes, véletlenszerű AppKey esetén a lánc már a crack-lépésnél elbukik. A teljes lánc (RF-sniff $\rightarrow$ offline crack $\rightarrow$ munkamenetkulcs-levezetés $\rightarrow$ forgalom visszafejtése) valódi hardveren és valós kriptográfiai műveletekkel igazolt, 2026-10-05-én.

7. Firmware-bináris elemzés és álcázott csomópont

7.1. Elv

A firmware-bináris az AppKey-t, DevEUI-t és AppEUI-t statikusan tárolja. Ha az eszközhöz fizikai hozzáféréssel rendelkezünk, a flash tartalmát a gyártó által biztosított eszközzel kiolvashatjuk (pl. bootloader módban). Firmware-szintű titkosítás nélkül a bináris plaintext; titkosítással védett esetén a debug interfészen keresztül olvasható, ha az nincs letiltva.

7.2. Kulcskinyerés a binárisból

Az eszköz két stratégiát alkalmaz:

1. **Közvetlen kulcskeresés (Strategy A):** az ismert AppKey bájt-mintájának mindkét bájt-sorrendben (LE/BE) keresése a binárisban. A firmware stack kulcstárolási elrendezésétől függően a DevEUI és AppEUI a megtalált kulcs közelében helyezkednek el.
2. **Entrópiaszkennelés (Strategy B):** a firmware binárisban szomszédos, magas entrópiájú 16 bájtos blokkok azonosítása. Gyártófüggetlen, különböző firmware stackeken is alkalmazható.

7.3. Álcázott csomópont generálása

A kinyert hitelesítő adatok (AppKey, DevEUI, AppEUI) alapján az eszköz automatikusan generál egy kész csomópont firmware-t (sablonból), amely:

- OTAA csatlakozást hajt végre az eredeti eszköz adataival,
- az áldozat payload-formátumát utánzó valósághű adatokat küld,
- opcionálisan WiFi+MQTT C2-csatornán keresztül futásidőben parancsokkal irányítható.

7.4. Eredmények és hatásvizsgálat

A generált rogue csomópont sikeresen regisztrált a ChirpStack hálózati szerveren az áldozat DevEUI alatt. A hálózati szerver naplójában az álcsomópont uplink csomagjai megkülönböztethetetlenek voltak az eredeti eszköz csomagjaitól. Az egyetlen detekciós lehetőséget a DevNonce visszajátszás-védelem nyújtotta, azonban ha az eredeti eszköz ki van kapcsolva, az álcsomópont zavartalan kommunikációt folytathat.

7.5. Munkamenetkulcs-levezetés érvényesítése éles teszttel

A fenti forgatókönyvet (fizikai hozzáféréssel korábban megszerzett AppKey felhasználása rogue csomópont klónozására) a hálózati oldalon is végigvittük: egy szabad fejlesztői panelt az áldozat eszköz valódi DevEUI/AppEUI/AppKey hármasával programoztunk, majd valódi OTAA Join-folyamatot hajtottunk végre a hálózati szerver ellen. A titkosított Join Accept választ – a valós RF- vagy SPI-elfogás gyakorlati helyettesítőjeként – a hálózati szerver saját, adminisztratív keretfolyam-API-ján keresztül rögzítettük, majd a levont munkamenetkulcsokat (NwkSKey, AppSKey) a klónozott eszköz saját AppKeyjéből, a Join Accept titkosított tartalmából és a DevNonce-ból vezettük le, a 2. fejezetben leírt képlet szerint.

A levezetés helyességének ellenőrzésére a legerősebb rendelkezésre álló próbát alkalmaztuk: a klónozott eszköz a soros konzoljára kiírta a saját, tőlünk teljesen független firmware-implementációja által kiszámolt DevAddr-t, amely **bit-pontosan megegyezett** az általunk, külső eszközből levezetett DevAddr-rel. Emellett a levezetett AppSKey-jel egy kontrollált titkosítás/visszafejtés kör (ismert plaintext → FRMPayload-titkosítás → visszafejtés) is helyesen adta vissza az eredeti adatot, megerősítve, hogy a payload-titkosítási lánc is helyesen lett reprodukálva.

Ez a teszt gyakorlatilag igazolja, hogy egy SPI-tappal vagy firmware-dumppal megszerzett AppKey birtokában a támadó – pusztán a hálózat felől megfigyelhető (vagy elfogható) Join-forgalom alapján – helyesen tudja reprodukálni mindkét munkamenetkulcsot, azaz a fenti álcázott csomópont nemcsak regisztrálni, hanem a titkosított adatforgalmat is hitelesen generálni/értelmezni képes.

8. Védelmi mechanizmusok értékelése

8.1. Firmware-szintű titkosítás

A firmware-szintű titkosítás bekapcsolása után az eszköz flash-tartalma titkosítva tárolódik; a titkosítási kulcs hardveresen védett és szoftveresen nem olvasható. Ennek következtében:

- **Hatékony:** közvetlen flash-dump (bootloader mód) plaintext kulcsot nem ad vissza.
- **Nem elégséges önmagában:** a futásidejű memória nem titkosított, az SPI-busz tap a Join Request plaintext keretét rögzíti, és ha a debug interfész nincs letiltva, azon keresztül a RAM kiolvasható. Az itt használt ESP32-platformon korábban már dokumentáltak is voltak olyan megvalósítási hibák (pl. CVE-2020-15048), amelyek a flash-titkosítást teljes egészében megkerülhetővé tették – ez is azt mutatja, hogy a védelem erőssége nem csak a bekapcsolt állapoton, hanem a konkrét implementáció helyességén is múlik [**?**].

8.2. Biztonságos indítás (Secure Boot)

A biztonságos indítás biztosítja, hogy csak aláírt firmware futhasson az eszközön, a gyártói referencia-implementáció szerint [**?**]. Ez megakadályozza a módosított/rogue firmware feltöltését, de önmagában nem véd a kulcskinyerés ellen, ha az eszköz fut. Hasonlóan a flash-titkosításhoz, a secure boot implementációja is hordozhat megkerülhető hibákat – ugyanezen a platformcsaládon korábban nyilvánosan dokumentált hiba (pl. CVE-2020-13629) tette lehetővé az aláírás-ellenőrzés megkerülését alkalmas firmware-képpel [**?**].

8.3. Megerősített firmware-konfiguráció

Az általunk fejlesztett megerősített konfiguráció – a biztonságos LoRaWAN-végpontokra vonatkozó szakirodalmi ajánlásokkal összhangban [**?**] – az alábbi intézkedéseket foglalja magában:

- firmware-szintű titkosítás + biztonságos indítás egyidejű bekapcsolása,
- debug interfész letiltása,
- erős, véletlenszerű 128 bites AppKey generálása provisioning során,
- DevNonce-kezelés: minden Join Request egyedi nonce, visszajátszásvédelem,
- alkalmazásszintű payload-titkosítás érzékeny szenzorértékekhez.

8.4. A debug-port lezárása olvasásvédelemmel (RDP) – hardveres igazolás

A fenti intézkedések közül a *debug interfész letiltását* valódi hardveren, a tesztkörnyezet ipari referencia-csomópontján (STM32WL55JC1) külön is igazoltuk, mivel ez zárja le közvetlenül a 7. fejezetben bemutatott firmware-kinyerési utat (a nyitott hibakeresési porton keresztüli flash- és memória-olvasást).

Kiindulás (RDP Level 0). Gyári állapotban az eszköz olvasásvédelme kikapcsolt (az `FLASH_OPTR` regiszter RDP-mezője `0xAA`). Ekkor a hibakereső a teljes flash-t korlátozás nélkül kiolvassa – a csomópont beégetett firmware-je és abban a kulcsanyag bitről bitre kinyerhető. Ez nem a hardver hibája, hanem *hardening-elmaradás*: a vizsgált gyári egység egyszerűen RDP Level 0-n került forgalomba.

Megerősítés (RDP Level 1). Az olvasásvédelmet Level 1-re emelve (RDP-mező `0xBB`) a hibakereső *csatlakozni* még tud, de a védett memóriából semmit nem olvas ki: a flash és a RAM olvasása egyaránt elutasításra kerül, sőt a mag regiszterei is kizárólag `0xFFFFFFFF` értéket adnak vissza (a megállított magon mért programszámláló `0xFFFFFFFE`). Lényeges, hogy a *csomópont funkcionálisan sértetlen marad*: a hardening életbe lépése után az eszköz újra csatlakozott a hálózathoz (OTAA-join, kiosztott DevAddr, növekvő felvételi keretszámláló) és küldte az uplinkeket – a védelem a kiolvasást zárja, nem a működést.

Visszaállíthatóság és a kulcsanyag védelme. Az olvasásvédelem Level 1-ről Level 0-ra csökkentése a hardver által kikényszerített *teljes flash-törlést* (mass-erase) vált ki: a művelet után a flash minden szava `0xFFFFFFFF`. Vagyis a védett tartalom nem

visszafejthető, kizárólag megsemmisíthető – a firmware és a benne tárolt kulcsok így nem szivárognak ki a hibakeresési porton.

Következtetés. A 7. fejezet firmware-kinyerési forgatókönyve tehát *egyetlen provisioning-lépéssel* (RDP Level 1 beállítása a gyártás végén) kivédhető, a csomópont működésének megtartása mellett. Ez megerősíti, hogy a vizsgált eszköz kiolvashatósága nem platform-szintű sebezhetőség, hanem konfigurációs hiányosság volt. (Az RDP Level 2 véglegesen, visszafordíthatatlanul tiltja a hibakeresést; ezt éles eszközön ritkán indokolt alkalmazni, és a jelen munkában tudatosan nem teszteltük.)

Korlát: a szoftveres olvasásvédelem feszültség-/óraglicheléssel megkerülhető. Fontos leszögezni, hogy az RDP Level 1 *szoftveresen* nyújt védelmet – más gyártók mikrokontrollerein (pl. STM32, Nordic nRF52) nyilvánosan dokumentált, feszültség- vagy óraglicheléssel (voltage/clock glitching) kivitelezett támadások ismertek, amelyek a hasonló célú olvasásvédelmi mechanizmusokat fizikai szinten kerülik meg, kifejezetten ezen bitek hardveres ellenőrzésének pillanatnyi megzavarásával [**? ? ?**]. Ez a jelen tesztkörnyezetben rendelkezésre álló eszközparkkal (alacsony költségű oszcilloszkóp, logikai analizátor) nem volt kivitelezhető, és a dolgozat keretein kívül eső, jövőbeli kutatási iránynak minősül (ld. 9. fejezet).

8.5. A védelmi szint összefoglalása

5. táblázat. Védelmi mechanizmusok hatékonysága a demonstrált támadásokkal szemben

Védelmi intézkedés	Passzív sniff	SPI-tap	Brute-force	Firmware-dump
(nincs)	–	–	–	–
Fw. titkosítás	–	–	–	✓
Fw. titkosítás + debug tiltás	–	–	–	✓✓
RDP Level 1 (olvasásvédelem)†	–	–	–	✓✓
Fw. titkosítás + biztonságos indítás	–	–	–	✓✓
Erős AppKey	–	–	✓	–
Teljes hardened konfig.	–	–	✓	✓✓

✓ = hatékony, ✓✓ = erős védekezés, – = nem nyújt védelmet

† valódi hardveren igazolva (STM32WL55JC1) – ld. az előző alfejezetet

Az SPI-busz tap és a passzív sniffing ellen szoftveres intézkedések önmagukban nem nyújtanak védelmet. A fizikai biztonsági intézkedések (tamper-evident tokozás, érintési érzékelők) szükségesek az SPI-tap fenyegetésének hatékony kezeléséhez.

	Passzív sniff	SPI-tap	Brute-force	Firmware-dump
(nincs)	–	–	–	–
Fw. titkosítás	–	–	–	✓
Fw. titkosítás + debug tiltás	–	–	–	✓✓
RDP Level 1 (olvasásvédelem)	–	–	–	✓✓
Fw. titkosítás + biztonságos indítás	–	–	–	✓✓
Erős AppKey	–	–	✓	–
Teljes hardened konfig.	–	–	✓	✓✓

5. ábra. A védelmi mechanizmusok hatékonysága a négy demonstrált támadással szemben (vizuális összefoglaló az előző táblázathoz: fehér = nem nyújt védelmet, világosszürke = hatékony, sötétszürke = erős védekezés).

8.6. Az OTA-frissítési csatorna hardeningje

A fenti intézkedések a végponti kulcsanyag védelmére irányulnak; a tesztkörnyezetben emellett a hálózati szerver és a végpontok közötti *firmware-frissítési* csatorna megerősítését is elvégeztük és valódi hardveren, éles ChirpStack-en keresztül ellenőriztük.

Tervezési alap. A LoRaWAN specifikáció (sem az 1.0.x, sem az 1.1-es verzió) nem ír elő OTA-frissítési biztonsági mechanizmust – ez kívül esik a protokoll hatókörén, a rádiós/Join-réteget szabványosítja, nem az alkalmazás-szintű firmware-disztribúciót. Az itt megvalósított aláírt manifeszt + anti-rollback + integritásvédelem hármas ezért nem egy LoRaWAN-specifikus előírást követ, hanem az IETF SUIT (Software Updates for IoT) munkacsoport jelenleg érvényben lévő, korlátozott erőforrású IoT eszközökre szabott architektúráját és manifeszt-információs modelljét alkalmazza erre a rétegre: aláírt/hitelesített és integritásvédett firmware-kép, szekvenciaszám-alapú anti-rollback-védelem,

valamint a frissítés metaadatait leíró manifesztum – pontosan a három elv, amit a tesztkörnyezetben implementáltunk és hardveren igazoltunk [**? ?**].

Teljes frissítési lánc külső flash-stagingen. A frissítési protokoll aláírt manifesztel indul, majd darabolt (chunk) adatátvitellel folytatódik munkamenet-azonosítóval és chunk-szintű CRC8-cal; a lezárás a teljes image SHA-256 ellenőrzése. Mivel a vizsgált integrált MCU+rádió eszköz 256 KB-os, egybankos belső flash-e nem elegendő egyszerre a futó és egy teljes staging-image tárolására, a frissítendő képet egy külső SPI NOR flash-re (W25Q64) stageljük. A teljes átvitelt több méretben igazoltuk hardveren: 250 B, 600 B (flash-lap határát átlépő) és 4500 B (több flash-szektort érintő) image, mindegyik hibátlanul, újraküldés nélkül. A vevő a stagelés lezárása után a képet *vissza is olvassa* a külső flash-ből, és ennek a tartalomnak a SHA-256-át veti össze a manifeszttel – így nem csupán a beérkezett bájtfolyam, hanem a ténylegesen flash-be írt tartalom helyessége is bizonyított.

A megvalósítás során egy, az integrált rádiós architektúrára jellemző, nem nyilvánvaló hibát is azonosítottunk és javítottunk: a flash-műveletek szinkron (blokkoló) végrehajtása – akár a rádiós megszakítás-kezelőben, akár a főciklusban – elegendő ideig feltartotta a LoRaWAN-protokollvermet ahhoz, hogy az eszköz lefagyjon. A megoldás egy nemblokkoló, a főciklusból lépkedő állapotgép, kiegészítve azzal a felismeréssel, hogy az alacsony fogyasztású STOP2 üzemmód nem őrzi meg a SPI-periféria konfigurációját, ezért azt minden ébredés után újra kell inicializálni.

6. táblázat. Hardveres OTA-átviteli mérések (NUCLEO-WL55JC1, külső W25Q-staging, nyugtázott downlink)

Image méret	Chunk	Újraküldés	Idő	Eredmény
250 B	5	0	~57 s	SHA-256 egyezés (visszaolvasva)
600 B	12	0	~137 s	SHA-256 egyezés (flash-lap határát átlépő)
4500 B	45	0	~476 s	SHA-256 egyezés (több flash-szektor)

Aláírás-ellenőrzés és anti-rollback. A megerősített (secure) tier a manifeszt Ed25519-aláírását ténylegesen ellenőrzi egy, a firmware-be ágyazott publikus kulccsal (auditált, a protokollverembe illesztett TweetNaCl implementációval); a számításigényes ellenőrzés a főciklusban fut, nem a valós idejű rádiós megszakítás-kezelőben. Három hardveres mérés igazolta a működést: érvényes aláírású kép elfogadása és stagelése; egyetlen bitjében rontott aláírás elutasítása (`signature INVALID`); valamint egy, a már elfogadottnál nem újabb verziószámú kép elutasítása (anti-rollback). Ez utóbbi egyúttal a verziószám belső flash-be való, újraindítás-tűrő perzisztálását is igazolta.

7. táblázat. A megerősített (secure) OTA-tier aláírás- és anti-rollback-döntései hardveren

Bemenet	Elvárt viselkedés	Mért eredmény
Érvényes aláírás, újabb verzió	elfogadás	stagelve, SHA-256 OK
Egy bitben rontott aláírás	elutasítás	`signature INVALID`
A perzisztáltnál nem újabb verzió	elutasítás	anti-rollback elutasítás

Sérülékeny és megerősített tier kontrasztja. A hardening hatása közvetlenül, azonos bemeneten mérhető. Ugyanazt az érvénytelen aláírású manifesztet az aláírást nem ellenőrző tier *elfogadja* és stageli, a megerősített tier *elutasítja*. Hasonlóan, a 6. fejezet AppKey-brute-force támadását egy szándékosan gyenge (alapértelmezett, csupa nulla) kulcsú végpont ellen lefuttatva a kulcs egy elfogott Join Requestből a másodperc töredéke alatt helyreállt, míg a véletlenszerű kulcsú, megerősített végpont ugyanazzal a támadással szemben ellenállt. Számszerűen: a gyenge kulcs a szótáras fázisban a másodperc töredéke alatt (˜0,01 s) helyreállt, míg a megerősített végpont kulcsa 20 szótár-jelölt után sem adódott – egy véletlenszerű 128 bites kulcs kimerítő keresése pedig gyakorlatilag kivitelezhetetlen.

Image-aktiváció – tápkimaradás-tűrő bootloader. A stagelt kép futóvá tételét (aktiváció) az egybankos flash miatt nem lehet hardveres A/B bank-váltással megoldani. Erre egy kisméretű saját bootloadert valósítottunk meg a flash elején, az alkalmazást pedig mögé helyeztük át (relokáció, a megszakítás-vektortábla áthelyezésével). A bootloader indításkor egy jelzőlapot olvas; ha frissítés van függőben, a külső flash-ből az alkalmazásrégióba másolja a képet, SHA-256-tal újraellenőrzi, és csak sikeres ellenőrzés után törli a jelzőt és adja át a vezérlést az új firmware-nek. Mivel a jelző csak az ellenőrzés után törlődik, a forráskép pedig a külső flash-en marad, egy másolás közbeni tápkimaradás a következő indításkor automatikusan újrapróbálódik – az eszköz nem válik működésképtelenné. A teljes láncot (aláírt kép fogadása, stagelés, aktiváció) és a szándékosan megszakított másolás utáni helyreállást is igazoltuk hardveren: az aktiváció után a belső flash tartalma bitre megegyezett a feltöltött képpel.

Munkamenet-izolláció – véletlenül reprodukált pozitív eset. Egy megszakadt, majd új munkamenettel újraindított átvitel során az eszköz helyesen utasította el egy korábbi, már nem aktív munkamenethez tartozó adatkeretek fogadását (`session mismatch, ignoring`) – ez a session-alapú izoláció tényleges, akaratlanul előidézett próbája volt, amely megerősítette, hogy idegen/elavult átviteli kísérletek nem tudják befolyásolni egy folyamatban lévő, más munkamenethez tartozó frissítést.

Kezelési csatorna: TLS + minimál jogosultságú API-kulcs. A frissítést vezérlő eszköz és a ChirpStack API közötti gRPC-csatorna korábban titkosítatlan volt annak ellenére, hogy az API-kulcs gyakorlatilag teljes hozzáférést biztosít. A vizsgált ChirpStack-verzió (v4.19.0) natív API-oldali TLS-t nem támogat, ezért a csatornát egy reverse proxy mögé helyeztük; a TLS-védett kapcsolatot végponttól végpontig ellenőriztük (tanúsítványlánc-validáció, hitelesített gRPC-hívás). Emellett az eddig használt, ideiglenes, teljes admin-jogú API-kulcsot egy állandó, tenant-szintre korlátozott kulcsra cseréltük, amelynek működését valódi, regisztrált eszközön is megerősítettük.

Kézbesítési megbízhatóság. Egy korábbi mérésben a hálózati szerver a downlink *kiküldését* (a várakozási sor kiürülését) jelezte sikeresként, nem az eszköz tényleges *vételét* – egy esetben 6/6 chunk sikeres kézbesítését jelezte, miközben az eszköz naplója szerint kevesebb érkezett meg. Ezt a hamis-pozitív jelzést nyugtázott (confirmed) downlinkek bevezetésével oldottuk meg, amivel a kézbesítés ténylegesen megbízhatóvá vált; a fenti, megszakítás nélküli teljes átvitelek már ezzel a mechanizmussal futottak.

9. Összefoglalás és jövőbeli kutatási irányok

9.1. Összefoglalás

Jelen dolgozatban négy LoRaWAN-specifikus támadási forgatókönyvet valósítottunk meg és értékeltünk egy kontrollált tesztkörnyezetben:

1. **Passzív SDR sniffing:** a Join Request teljes metaadatkészlete (AppEUI, DevEUI, DevNonce, MIC) hálózati hozzáférés nélkül rögzíthető olcsó, kereskedelemben elérhető eszközzel.
2. **SPI-busz tap:** fizikai hozzáféréssel a LoRaWAN keret az adás előtt plaintext formában kiolvasható a rádióchip FIFO-írásából, firmware-szintű titkosítás bekapcsolt állapotában is.
3. **AppKey brute-force:** gyenge vagy alapértelmezett kulcsok esetén a rögzített Join Request MIC alapján az AppKey másodpercek-percek alatt helyreállítható offline brute-force-szal.
4. **Firmware-elemzés és rogue csomópont:** flash-dumpból az AppKey, DevEUI és AppEUI automatizáltan kinyerhető; a generált álcsomópont megkülönböztethetetlenül kommunikál a hálózattal az eredeti eszköz identitásával.

Az eredmények megerősítik, hogy a LoRaWAN kriptográfiai védelme erős, de a fizikai eszközbiztonság és a kulcsmenedzsment gyengeségei szignifikáns kockázatot jelentenek a

valós telepítésekben.

A támadások mellett a megfelelő ellenintézkedéseket is megvalósítottuk és hardveren igazoltuk (8. fejezet), így a hardening hatása nem csupán elvben, hanem mérhetően is kimutatható. Kiemelten: az integrált rádiós (STM32WL-osztályú) végponton felépítettünk egy teljes, biztonságos firmware-frissítési láncot – külső SPI-flash alapú staging, aláírt (Ed25519) manifeszt, anti-rollback, a flash-ből visszaolvasott tartalom integritásellenőrzése, és egy tápkimaradás-tűrő saját bootloader az aktiváláshoz –, amelyet az aláírt kép fogadásától a telepítésig végig igazoltunk. A védelem hatékonyságát pozitív kontrollal mértük: ugyanaz a támadás (érvénytelen aláírású kép, illetve gyenge kulcs elleni AppKey-brute-force) a szándékosan gyenge konfigurációjú végpontot kompromittálja, a megerősített, azonos platformú végpont viszont ellenáll. Ez közvetlen, azonos bemeneten mért bizonyíték arra, hogy a feltárt kockázatok a szokásos kulcs- és frissítés-hardeninggel ténylegesen kezelhetők.

9.2. Kiegészítő eredmények: hálózatszerver- és keretszintű vizsgálatok

A fő négy forgatókönyv mellett három, opportunista jellegű kiegészítő vizsgálatot is lezártunk, amelyek nem a LoRaWAN-protokollt, hanem a hálózati szerver (ChirpStack) és a kulcs nélküli RF-felület határait térképezik fel.

ChirpStack eszközprofil-kódek, erőforrás-kimerítés: egy payload-vezérelt, szándékosan végtelen ciklust tartalmazó kódek-script nem okozott erőforrás-kimerítési (DoS) hatást a hálózati szerveren – a vizsgált ChirpStack 4.19.0 a kódek-futásidőt korlátozza, időtúllépés esetén a dekódolt objektum `null` lesz, a szerver válaszképes marad. Negatív eredmény, de a korlátozás hiányában a kockázat valós lenne.

Kódek-injekció a nyelőig: ugyanazon kódek-rétegen keresztül egy célzottan összeállított on-wire string sanitizálatlanul, verbátim eljut egy MQTT-integráción át egy mintaszerű üzemeltetői webes irányítópultig. Egy `innerHTML`-alapú DOM-nyelőn keresztül ez markup-injekcióhoz, weaponizált payloaddal pedig script-végrehajtáshoz vezet az üzemeltető böngésző-originjében (tárolt XSS); `textContent` használata a nyelőnél megszünteti a sebezhetőséget. A hiba súlyossága alkalmazásrétegi (saját kódek és irányítópult-implementáció), nem a ChirpStack magjáé.

Kulcs nélküli RF-outsider sweep: egy rendesen, véletlenszerű AppKey-jel kulcsolt eszköz esetén egy kulcs nélküli, csak RF-hozzáféréssel rendelkező támadó nem tud kulcsot vagy munkamenetadatot kinyerni (ld. a 6. fejezet negatív kontrollja). Egyetlen, alacsony súlyosságú találat maradt: a hálózati szerver nem korlátozza a Join Request-ek rátáját,

ezáltal egy folytonos, véletlen azonosítójú Join-flood elérhetőségi (availability) kockázatot jelenthet, hitelesítés vagy emberi beavatkozás nélkül. Két további, csomagformátum-határt feszegető variáns (minimális hosszúságú és csonkított MIC-ű keret) tiszta negatív eredményt adott – a vizsgált LoRaWAN-keretelemző robusztusan kezelte ezeket.

9.3. Jövőbeli kutatási irányok

- **LoRaWAN 1.1 biztonsági értékelése:** a DevNonce-kezelés és a JoinEUI mechanizmus vizsgálata a LoRaWAN 1.1-es specifikáció alapján.
- **Bit-flipping és MIC-hamisítás:** NwkSKey nélküli csomagmódosítás határainak vizsgálata érzékeny ipari alkalmazásokban.
- **Hálózati szintű anomáliadetekcó:** gépi tanulás alapú rendellenesség-detekció megvalósítása ChirpStack platformon a rogue csomópont detekciójára.
- **Több eszközcsalád vizsgálata:** a megvalósított eszközkészlet kiterjesztése más LoRaWAN-eszközökre (RAK4631, TTGO T-Beam, Seeed LoRa-E5).
- **Tamper-védelem hatékonysága:** fizikai tamper-érzékelők (accelerometer-alapú, fénydetektor) integrálásának vizsgálata az SPI-tap ellen.
- **Feszültség-/óraglicheléses eszközbeszerzés:** a 8. fejezetben említett, más gyártói platformokon dokumentált glitching-alapú RDP-megkerülés ([**?** **?** **?**]) tényleges kipróbálása a STM32WL-en, dedikált glitch-hardver (pl. ChipWhisperer-Nano/Husky, PicoGlitcher) beszerzése esetén.
- **Hálózati szerver háttérrendszerek nagyléptékű biztonsági felmérése:** a jelen dolgozat egyetlen, kontrollált ChirpStack-példányra fókuszál; nagyszámú, éles IoT-háttérrendszer (MQTT/CoAP/XMPP-alapú) biztonsági állapotát vizsgáló nagyléptékű mérések rámutatnak, hogy az itt talált konfigurációs hiányosságok (pl. gyenge AppKey, hiányzó rate-limit) iparági léptékben is tipikusak [**?**].
- **Automatizált LoRaWAN biztonsági tesztelő keretrendszer:** a jelen munkamenetben kézzel összeállított PoC-ok helyett egy dedikált, újrafelhasználható tesztelő keretrendszer (pl. a ChirpOTLE mintájára) bevezetése rendszeresebb regressziós biztonsági teszteléshez [**?**].